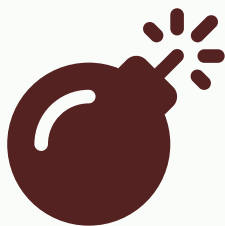


Backend reference

Node.js API for Smart AgriTech EMS: `ems/ems-backend/`.

Request pipeline



Syntax error in text
mermaid version 10.9.6

Mount order (`server.js`):

- 1. `GET /health`, `GET /metrics`
- 2. `POST /api/ingest/*` (no JWT)
- 3. `app.use('/api', apiLimiter, routes)`

Authentication & authorization

JWT auth (`middleware/auth.js`)

Header		Format
Authorization		Bearer <accessToken>

Role	Enum	Typical guards
Super Admin	SUPER_ADMIN	authorize('SUPER_ADMIN')
Org Admin	ORG_ADMIN	authorize('SUPER_ADMIN', 'ORG_ADMIN')
User	USER	Device-scoped in controllers

Ingest auth (`utils/ingestAuth.js`)

Header	Value
<code>x-api-key</code>	Global <code>INGEST_API_KEY</code> or per-device key (SHA-256 hash in DB)

Per-device keys returned **once** on `POST /api/devices` and `POST /devices/:id/regenerate-ingest-key`.

API route map

Auth — `/api/auth`

Method	Path	Auth
POST	<code>/login</code>	Public (rate limited)
POST	<code>/refresh</code>	Public
POST	<code>/logout</code>	Public
GET	<code>/me</code>	JWT
POST	<code>/change-password</code>	JWT
POST	<code>/forgot-password</code>	Public
POST	<code>/reset-password</code>	Public

Ingest — `/api/ingest`

Method	Path	Body
POST	<code>/</code>	<code>{ deviceId, slaveId?, readings[] }</code>
POST	<code>/command-ack</code>	<code>{ deviceId, commandId, status?, reason? }</code>

Devices — `/api/devices`

Method	Path	Roles (mutations)
GET/POST	<code>/</code>	POST: SUPER_ADMIN, ORG_ADMIN
GET/PUT/DELETE	<code>/:id</code>	PUT/DELETE: SUPER_ADMIN, ORG_ADMIN
PATCH	<code>/:id/switch</code>	SUPER_ADMIN, ORG_ADMIN

Method	Path	Roles (mutations)
POST	<code>/:id/regenerate-ingest-key</code>	SUPER_ADMIN, ORG_ADMIN
GET	<code>/:id/commands/:commandId</code>	Authenticated

Device config — `/api/devices/:deviceId/config` :

- `GET /slaves` , `GET /slaves/:id/variables`
- `PATCH /variables/:id` , `GET /variables/:id/log`

Device users — `/api/devices/:deviceId/users` :

- `GET` , `POST` , `DELETE /:userId`

Gateways — `/api/gateways`

Full CRUD; mutations: SUPER_ADMIN, ORG_ADMIN.

Device templates — `/api/device-templates`

- CRUD + `POST /:id/clone`
- Nested: `/slaves` , `/slaves/:slaveId/variables` CRUD

Sensor data — `/api/sensor-data`

Method	Path	Purpose
GET	<code>/latest</code>	Current values per variable (<code>?deviceId&slaveId</code>)
GET	<code>/history</code>	Single variable time series (<code>variableName</code> required)
GET	<code>/readings</code>	Paginated raw ingest rows
GET	<code>/aggregate</code>	Bucketed averages (<code>variableName</code> , <code>timeRange</code>)
GET	<code>/dashboard-summary</code>	KPIs + chart data for dashboards
GET	<code>/download</code>	CSV export
DELETE	<code>/</code>	SUPER_ADMIN bulk delete

Time ranges: `24h` , `7d` , `30d` (see `utils/helpers.js` `TIME_RANGE_MS`).

AI analytics — `/api/ai`

Path	Purpose
<code>/energy-consumption</code>	Consumption charts

Path	Purpose
<code>/power-factor</code>	PF trend + alarms
<code>/voltage-imbalance</code>	Phase voltage analytics
<code>/current-imbalance</code>	Current analytics
<code>/predictions</code>	Forecast-style readings

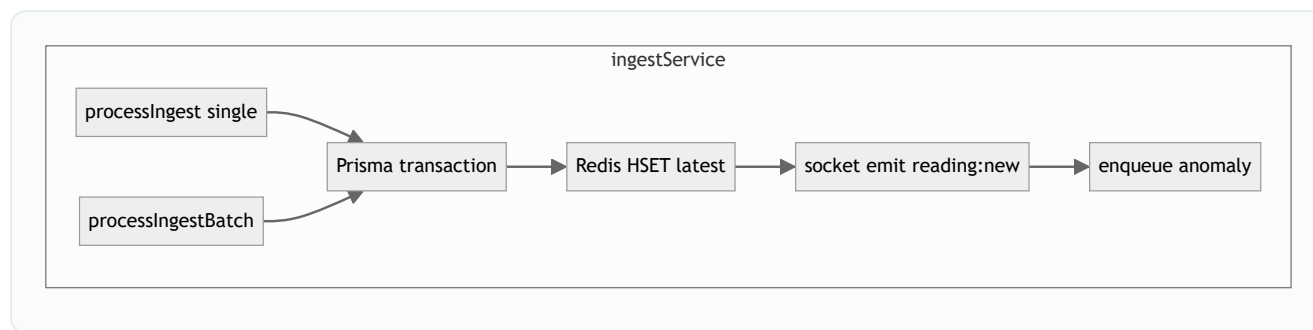
Alarms — `/api/alarm-*`, `/api/alarm-history`

- Templates, settings, contacts: CRUD
- History: variable alarms, linkage records, notifications
- `PATCH /alarm-history/variable-alarms/:id/process`

Other groups

Prefix	Features
<code>/api/organizations</code>	CRUD; <code>/me</code> for org admin
<code>/api/users</code>	CRUD, status, reset password
<code>/api/notifications</code>	List, read, delete
<code>/api/scheduled-tasks</code>	CRUD, toggle, logs
<code>/api/anomalies</code>	List, timeline, acknowledge
<code>/api/interval-history</code>	Billing intervals
<code>/api/slab-rates</code>	Tariff slabs
<code>/api/widget-templates</code>	Dashboard widgets
<code>/api/icons</code> , <code>/products</code> , <code>/themes</code> , <code>/settings</code>	Platform assets
<code>/api/subscriptions</code>	Plan requests
<code>/api/device-timestamps</code>	Connectivity log

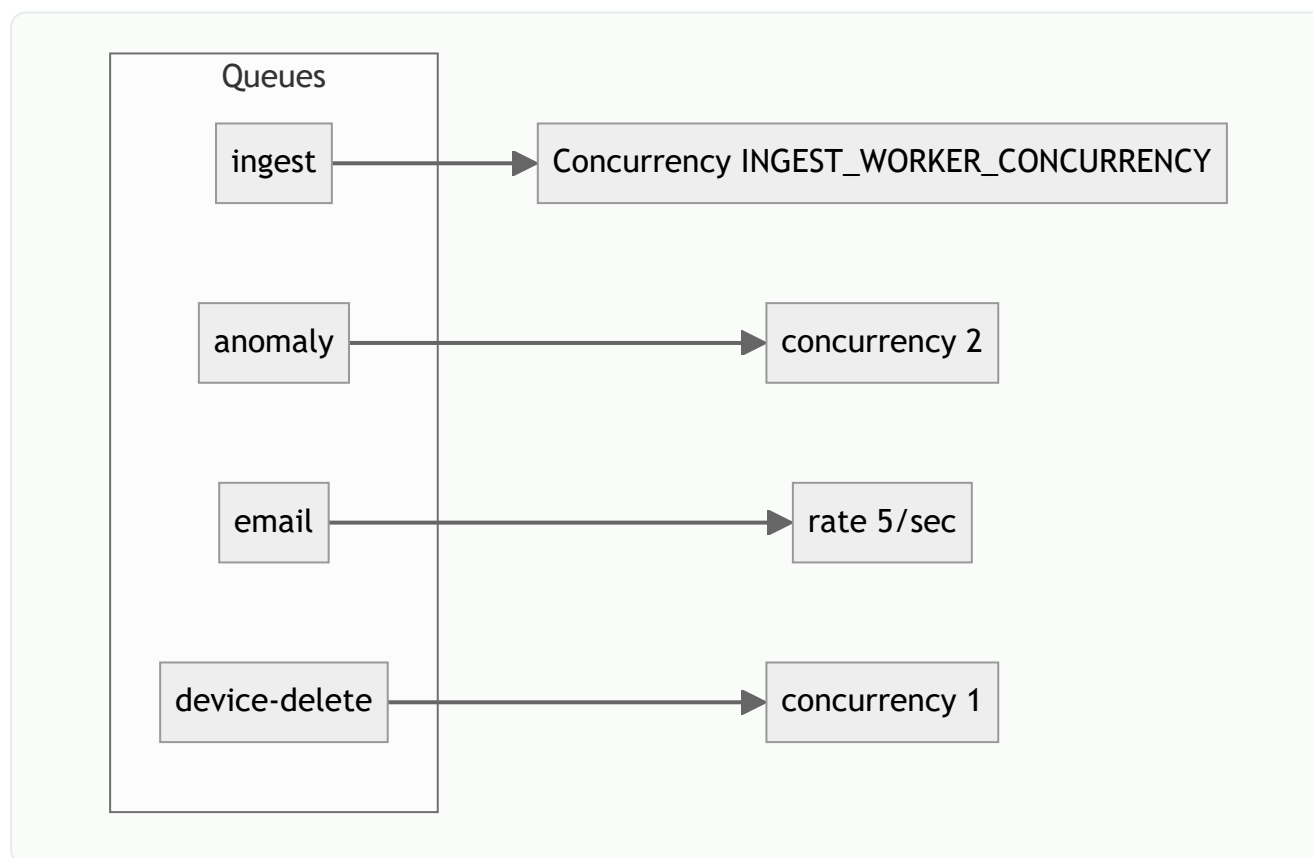
Ingest service (deep dive)



Writes per ingest:

1. `SensorReading` — JSON blob of all variables in batch
2. `SensorReadingValue` — normalized float rows (analytics indexes)
3. `Device.lastDataReceivedAt` , `DeviceTimestamp`
4. `DeviceConfigVariable.currentValue` (unless `SKIP_PG_CURRENT_VALUE=true`)
5. Redis `device:{id}:latest` hash

Workers (`workers/jobQueues.js`)



Queue	Trigger	Handler
<code>ingest</code>	HTTP ingest when Redis up	<code>processIngestBatch</code>
<code>anomaly</code>	After ingest	<code>runAnomalyCheck</code>
<code>email</code>	Alarm notification	Nodemailer send
<code>device-delete</code>	Device removal	Cascade purge

Socket.IO (`socket/index.js`)

Server → client events

Event	Room	Payload
<code>reading:new</code>	<code>device_{deviceId}</code>	<code>{ deviceId, ... }</code>
<code>device:command</code>	<code>device_{deviceId}</code>	Command status
<code>device:switch</code>	<code>org_{organizationId}</code>	Scheduler switch
<code>alarm:new</code>	<code>org_{organizationId}</code>	Alarm summary

Client → server

Event	Action
<code>join:device</code>	Subscribe to device room

Auth: `handshake.auth.token` must be valid JWT.

Database model summary

Domain	Models
Tenancy	<code>Organization</code> , <code>User</code> , <code>RefreshToken</code> , <code>Subscription</code>
IoT	<code>Gateway</code> , <code>Device</code> , <code>DeviceTemplate*</code> , <code>DeviceConfig*</code> , <code>DeviceUser</code> , <code>MqttConfig</code>
Telemetry	<code>SensorReading</code> , <code>SensorReadingValue</code> , <code>AIForecastReading</code>
Commands	<code>DeviceCommand</code>
Alarms	<code>TemplateTrigger</code> , <code>AlarmSetting</code> , <code>AlarmContact</code> , <code>DeviceVariableAlarmHistory</code> , <code>DeviceVariableLinkageHistory</code>

Domain	Models
Ops	<code>ScheduledTask</code> , <code>ScheduleExecutionLog</code> , <code>Notification</code>
Billing	<code>SlabRate</code> , <code>IntervalHistory</code>
UI	<code>WidgetTemplate</code> , <code>Icon</code> , <code>Theme</code> , <code>Product</code> , <code>SystemSetting</code>

Schema: `prisma/schema.prisma`

Migrations: `prisma/migrations/`

Environment variables

Variable	Required	Description
<code>DATABASE_URL</code>	Yes	PostgreSQL connection
<code>JWT_SECRET</code>	Yes	Token signing
<code>CLIENT_URL</code>	Yes prod	CORS origins (comma-separated)
<code>REDIS_URL</code>	Prod recommended	Queued ingest + cache
<code>INGEST_API_KEY</code>	Yes	Global ingest fallback
<code>PORT</code>	No	Default 5000 (CapRover: 9001)
<code>SKIP_PG_CURRENT_VALUE</code>	No	Default <code>true</code> with Redis
<code>INGEST_BATCH_MAX</code> / <code>INGEST_BATCH_MS</code>	No	Batch tuning
<code>INGEST_DEVICE_MAX_PER_MIN</code>	No	Per-device rate limit
<code>VALUE_FLUSH_MS</code>	No	Redis→PG flush interval
<code>DATABASE_READ_URL</code>	No	Read replica
<code>CLOUDINARY_*</code>	No	Image uploads
<code>SMTP_*</code> / <code>NODEMAILER_*</code>	No	Email
<code>PRISMA_LOG_QUERIES</code>	No	Debug only

Full list: `ems/ems-backend/.env.example`

Scripts & tooling

npm script	Purpose
<code>npm run dev</code>	Nodemon server
<code>npm run seed</code>	Demo users & sample data
<code>npm run seed:fleet</code>	100 devices / 10 gateways
<code>npm run simulate:production</code>	1s interval fleet simulator
<code>npx prisma migrate deploy</code>	Apply migrations

Docker

```
# ems/ems-backend/Dockerfile
# - node:20-alpine
# - prisma generate + migrate on entrypoint
# - EXPOSE 5000 (or CapRover PORT)
```

`docker-entrypoint.sh` runs `npx prisma migrate deploy` before `node server.js`.

Health & metrics

`GET /health`

```
{
  "status": "ok",
  "redis": true,
  "ingestMode": "queued"
}
```

`GET /metrics` — Prometheus text format (`utils/metrics.js`).

Related documents

- [Architecture](#)
- [System flows](#)
- [Web frontend](#)

- [Deployment](#)